



Hackathon Challenge — Build an Agentic Tax-Filing Assistant

Mission

Build a small **agentic system** that helps a person file a U.S. federal income tax return (Form 1040) by chatting with them. A user shows up with a single W-2 from a job paying roughly **\$40,000/year**, has a short, friendly conversation, and walks away with a **completed 2025 Form 1040 they can download**.

You are given two things: the **four pillars** your harness must demonstrate, and the **end result** it must produce. **Everything in between is your decision**. Part of the challenge — and part of how you'll be judged — is the architecture you choose to get from the pillars to the result.

The Four Pillars (required)

Your agent **must** be built on a harness that clearly demonstrates all four. How you implement, enforce, and expose each one is up to you — but a judge should be able to point at your code and your running system and see each pillar working.

1. **Chat loop** — a conversational loop between the user and the agent that carries state across turns.
2. **Tools** — the agent can take real actions through defined tools, not just talk. At minimum, something that produces the filled return.
3. **Guardrails** — constraints that keep the agent on-task, safe, and bounded (what it will and won't do, validation of what it accepts, limits it respects).
4. **Observation** — the agent's behavior is observable: you (and a judge) can see what it did, the decisions it made, and the actions it took.

You decide what each pillar concretely means in your design and how to make it legible. "It's in the prompt" is weaker than "it's enforced and visible."

The End Result (definition of done)

A working system where:

- ☐ There is a **web-based chat** a user can interact with.
 - ☐ The user provides a **W-2 for a person earning ~\$40,000/year** (you supply a realistic fake one for testing).
 - ☐ The agent asks **no more than 5 questions** to gather what it needs.
 - ☐ The conversation feels **warm and human** — friendly, clear, not robotic or interrogative. Quality of communication is explicitly part of the bar.
 - ☐ The agent fills out a **2025 IRS Form 1040** based on the W-2 and the user's answers.
 - ☐ The completed 1040 can be **downloaded** as a file when finished.
 - ☐ The system is **deployed** to a public URL — on **Render or a comparable free, easy hosting service** — that a judge can reach and try.
-

Fixed constraints (non-negotiable)

These are part of the problem. Don't change them.

- **Tax form:** U.S. Federal **Form 1040, tax year 2025**.
 - **Taxpayer profile:** W-2, ~\$40,000/year earner.
 - Must be able to change inputs based on single, married filing status, etc.
 - **Question budget:** **5 questions** asked of the user.
 - **Tone:** genuinely friendly, human-quality conversation.
 - **Output:** a **downloadable completed form**.
 - **Interface:** a **web chat**.
 - **Deployment:** publicly reachable at a live URL, hosted on **Render or a comparable free, easy-to-use service**.
 - **It must actually work** end-to-end, not just demo a happy-path mock of one step.
-

Your decisions (intentionally open — choose and justify)

This is the design space. Pick what gets you to the end result best, and be ready to explain why:

Language & framework — anything.

LLM / model / provider — anything (or none, if you can justify it).

How you obtain and fill the 1040 — how the form is acquired, how fields get populated, how the file is produced.

Where the W-2 comes from — how the user supplies it and how the agent reads it.

Tax computation — how accurate, and how you compute it.

How guardrails are enforced — prompt, code, schema, or a mix.

Conversation design — which 5 questions, in what order, and how you keep it human.

State & sessions — how conversation state is held.

Hosting / deployment — which free service you deploy to (Render or a comparable free, easy option).

Testing — how you prove it works.

If you find yourself blocked on a "but the spec doesn't say X" — that's the point. Make a reasonable call, document it, move on.

How you'll be judged

In priority order:

- **Harness quality (highest weight)** — how cleanly and convincingly the four pillars are realized. Are they real and enforced, or cosmetic?

- **Does it actually work** — a real W-2 in, a real downloadable 1040 out, via the chat, end-to-end.
- **Conversation quality** — does it feel like a helpful human, within the 5-question budget?
- **Soundness of your decisions** — the choices you made for the open items above, and how well you can defend them.

Not judged: visual/UI polish. Keep the front end minimal. Spend your effort on the harness and on making it work.

Deliverables

- **Source code** in a repository.
- **A deployed, running system** a judge can try — live at a public URL, hosted on Render or a comparable free, easy-to-use service. Include the URL plus one-command local run instructions as a fallback (not a substitute).
- **A short DECISIONS note** (half a page is fine): the key choices you made for the open items, and why. This is where you show your reasoning.

Rules & scope

- Keep it **simple**. This is a prototype, not a product. Resist scope creep — breadth of features is not the goal; a working, well-architected harness is.
- A **fake** W-2 and test data only. No real PII, no real filings, no e-filing.
- This is an educational/hackathon exercise, **not tax advice** — and your agent shouldn't pretend to give it.

Optional stretch goals (only if the core is done and solid)

- Handle a second filing status or a dependent gracefully.
- Let the user correct an answer mid-conversation.
- Surface the observation trail in the UI, not just the logs.
- Validate the W-2 input and recover from messy/partial data.

Reminder

You have the **pillars** and the **destination**. The route is yours to design.
Make deliberate choices, keep it working, and keep it simple.

Rapidly assess new content, devise a plan, new agents get things done

Pillars 4: guardrails, observability, tools, harness

Harness; car; multiturn looping, validate inputs outputs, guardrails, observability

Model; engine;

The following criteria matter when building:

- Ability to repeat, maintain history, add information in loop
- Tools - function to add info to llm, schema, executor,
- Guardrail (keep it in bounds) - input, action, output
- Observability - p95 latency, \$/run, err%, eval

Agent will be able to Fill out the 1040 with a sample w2 and get the actual and correct results as is filing with the government.

This will be deployed on render.

W2s to upload will be in pdf format.

Please look at 1040.pdf and look into how and what needs to be filled out, calculated, and how. Also look at samplew2.pdf to see what a sample w2 form would look like, and the information needed to fill out a 1040 form.

Reference the proactive and on-demand agent in

"C:\Users\jaynyasg\OneDrive\Documents\GitLab\ac" for agent build. Please be aware of the ai costs and analysis and guidelines.

Reference "C:\Users\jaynyasg\OneDrive\Documents\GitLab\Taxes" for other tax filing repos to see how they get their information and build their agents. Our agent will need tax knowledge, tax filing knowledge, and 1040 knowledge.

Reference "C:\Users\jaynyasg\OneDrive\Documents\GitLab\house-style" for design.